

Assets, OAuth, JWT, SOS

Token Identifiers · String Concepts · LLM Tokenisation Strategies

PART 1 — Assets in Software Systems

What is an Asset?

An asset is any resource with value in a software system — data, files, credentials, keys, or intellectual property that must be protected.

- Digital assets: API keys, tokens, certificates, user data, media files.
- Infrastructure assets: servers, databases, cloud storage buckets.
- Assets are classified by sensitivity: public, internal, confidential, restricted.
- Asset management = inventory + ownership + access control + lifecycle.

Asset Classification Example: PUBLIC → marketing images, public docs INTERNAL → internal APIs, employee data CONFIDENTIAL → PII, payment info, secrets RESTRICTED → private keys, root credentials

Static vs Dynamic Assets

Static assets are fixed files served as-is; dynamic assets are generated on-the-fly per request.

- Static: HTML, CSS, JS bundles, images, fonts — served via CDN.
- Dynamic: personalised dashboards, reports, user-specific API responses.
- Static assets use Cache-Control headers and content hashing for cache busting.
- CDNs (CloudFront, Fastly, Cloudflare) distribute static assets globally.

Cache-busting via content hash main.abc123.js ← hash changes when file changes #
Cache-Control header Cache-Control: public, max-age=31536000, immutable

Asset Security — Secrets Management

Secrets (API keys, DB passwords, tokens) must never be hardcoded in source code — use a secrets manager.

- Never commit .env files to version control.
- Use: AWS Secrets Manager, HashiCorp Vault, Azure Key Vault.
- Rotate secrets regularly; revoke on team member departure.
- Scan repos for leaked secrets: GitGuardian, truffleHog, git-secrets.

Bad — hardcoded secret DB_PASSWORD = 'super_secret_123' # Good — from environment / vault
import os DB_PASSWORD = os.getenv('DB_PASSWORD') # or fetched at runtime from Vault/AWS SSM

PART 2 — OAuth 2.0 Deep Dive

What is OAuth 2.0?

OAuth 2.0 is an open authorization framework that lets a third-party application access a user's resources without exposing their credentials.

- Delegates access — user never gives password to the app.
- Issues scoped, time-limited access tokens instead.
- 4 core roles: Resource Owner, Client, Authorization Server, Resource Server.
- Used by Google, GitHub, Facebook, Twitter for social login.

Roles: Resource Owner → The user (you) Client → The app wanting access Auth Server → Issues tokens (Google, Auth0) Resource Server → API holding protected data

■ OAuth 2.0 handles AUTHORIZATION (what you can do), not authentication (who you are). OIDC adds authentication on top.

OAuth 2.0 Grant Types

Grant types define how the client obtains an access token depending on the application architecture.

- Authorization Code + PKCE — most secure; for web & mobile apps.
- Client Credentials — machine-to-machine; no user involved.
- Device Code — for TV / CLI apps without a browser.
- Implicit (deprecated) — tokens in URL; replaced by Auth Code + PKCE.

Authorization Code Flow (simplified): 1. User clicks 'Login with Google' 2. Redirect to Google /authorize?scope=email 3. User consents → Google returns code 4. App exchanges code for access_token + refresh_token 5. App calls API with Bearer access_token

Access Token vs Refresh Token

Access tokens are short-lived credentials for API calls; refresh tokens obtain new access tokens without re-login.

- Access token: short-lived (15 min – 1 hr), sent with every API request.
- Refresh token: long-lived (days–weeks), stored securely, never sent to APIs.
- Refresh token rotation: issue a new refresh token on each use, invalidate old.
- Store refresh tokens in HTTP-only cookies, never localStorage.

Token lifetimes (typical) access_token → expires_in: 3600 (1 hour) refresh_token → expires_in: 2592000 (30 days) # Refreshing POST /oauth/token grant_type=refresh_token refresh_token=

OAuth Scopes

Scopes define the specific permissions granted to a client — the principle of least privilege applied to API access.

- Request only the scopes you need.
- Examples: read:user, write:posts, email, openid, profile.
- Users see and consent to scopes on the authorization screen.
- Resource server validates the scope on each API request.

Requesting scopes GET /authorize ?scope=openid profile email read:orders &client_id=my_app &redirect_uri=https://app.com/callback &response_type=code

PART 3 — JWT (JSON Web Tokens)

What is a JWT?

A JSON Web Token is a compact, URL-safe, self-contained token that encodes claims as a JSON object and is cryptographically signed.

- Structure: Header.Payload.Signature (3 Base64url parts separated by dots).
- Stateless — server needs no database lookup to validate.
- Signed with: HS256 (symmetric) or RS256/ES256 (asymmetric key pair).
- Used as access tokens in OAuth 2.0 and OpenID Connect.

JWT Structure: eyJhbGciOiJSUzI1NiJ9 ← Header (algo + type) .eyJ1c2VyX2lkIjoiaW5kaWQ== ← Payload (claims) .SflKxwRJSMeKKF2QT4fww... ← Signature (verification)

- **JWTs are encoded, NOT encrypted.** Anyone can decode the payload — never put passwords or sensitive PII in a JWT unless using JWE (JSON Web Encryption).

JWT Claims

Claims are statements about the subject (user) embedded in the payload — registered, public, or private.

- Registered claims (standard): iss, sub, aud, exp, iat, nbf, jti.
- iss = issuer, sub = subject (user ID), exp = expiry timestamp.
- aud = audience (intended recipient), jti = JWT unique ID (prevents replay).
- Private claims: custom app data like roles, permissions, tenant_id.

```
{ "iss": "https://auth.myapp.com", "sub": "user_42", "aud": "https://api.myapp.com", "exp": 1750000000, "iat": 1749996400, "roles": ["admin", "editor"], "jti": "unique-token-id-abc123" }
```

JWT Validation

Validating a JWT means verifying its signature, expiry, and claims — not just decoding it.

- Verify signature using the issuer's public key or shared secret.
- Check exp — reject expired tokens.
- Check aud — ensure the token is meant for your service.
- Check iss — ensure the token came from the expected issuer.
- Maintain a jti blocklist to reject revoked tokens.

```
# Node.js JWT validation import jwt from 'jsonwebtoken'; const payload = jwt.verify(token, PUBLIC_KEY, { algorithms: ['RS256'], audience: 'https://api.myapp.com', issuer: 'https://auth.myapp.com', }); // Throws if invalid, expired, or wrong audience
```

JWT Security Best Practices

JWTs are powerful but misuse leads to critical vulnerabilities.

- Reject "none" algorithm — attackers can forge unsigned tokens.
- Use RS256 (asymmetric) over HS256 when multiple services verify tokens.
- Keep access JWTs short-lived ($\leq 15\text{--}60$ min); use refresh tokens for longevity.
- Store in memory or HTTP-only cookies — NOT localStorage (XSS risk).
- Rotate signing keys periodically; support key ID (kid) in headers.

```
// Always specify allowed algorithms – never trust header alone jwt.verify(token, key, {
algorithms: ["RS256"] }); // Bad – vulnerable to algorithm confusion jwt.verify(token, key);
// accepts "none" in some libs!
```

PART 4 — SOS: State, Object & Session Management

State Management

State is any data that persists across operations. Managing it correctly is critical to application consistency and security.

- Client state: UI state, form data, in-memory caches (React useState, Redux).
- Server state: DB records, distributed cache (Redis), message queues.
- Shared state across services needs a single source of truth.

- Stateless APIs (REST) push state to the client (JWT) or a shared store.

State storage options by use-case: Short-lived / ephemeral → In-memory (Redis, Memcached)
 User session → Redis session store Long-term / persistent → PostgreSQL, MongoDB Client-side
 → HTTP-only cookie, memory (not localStorage)

Object Management & Serialisation

Objects (data structures) must be serialised to cross service / network boundaries — JSON, Protobuf, MessagePack are common formats.

- JSON: human-readable, universal, but verbose and slow for large payloads.
- Protobuf (Protocol Buffers): binary, 3–10x smaller and faster than JSON.
- MessagePack: binary JSON — compact, widely supported.
- Always validate/sanitise deserialised objects before use.

JSON vs Protobuf payload size (same data) JSON: { id: 42, name: 'Ada' } → ~22 bytes
 Protobuf: binary encoded → ~8 bytes # Python serialisation import json, pickle
 json.dumps(obj) # Safe, cross-language pickle.dumps(obj) # Fast, Python-only – NEVER
 deserialise untrusted pickle!

Session Management

A session maintains user context across multiple HTTP requests, since HTTP itself is stateless.

- Server-side sessions: ID stored in cookie, data in Redis/DB — revocable.
- Client-side sessions: JWT in cookie — stateless, not easily revocable.
- Set Secure, HttpOnly, SameSite=Strict on session cookies.
- Implement idle timeout and absolute session expiry.
- Regenerate session ID after login to prevent session fixation attacks.

Secure session cookie attributes Set-Cookie: sessionId=abc123; HttpOnly; # No JS access →
 prevents XSS theft Secure; # HTTPS only SameSite=Strict; # No cross-site sending → prevents
 CSRF Max-Age=3600; # 1-hour expiry Path=/;

PART 5 — Token Identifiers & String Concepts

What is a Token Identifier?

A token identifier is a unique string or value used to identify, authenticate, or authorize an entity in a system.

- Auth tokens: JWT, opaque bearer tokens, API keys, session IDs.
- Opaque tokens: random strings (UUID/ULID) with no embedded data — decoded server-side.
- Structured tokens: JWT — self-contained with claims readable by any party.
- Token ID (jti) uniquely identifies a specific token instance for revocation.

Token types compared: Opaque token → 'eyJXk9mQ3pZ...' (random, looked up in DB) JWT →
 header.payload.sig (self-contained, verified by sig) API Key → 'sk_live_abc...' (static,
 identifies an app) Session ID → '8f3a9...' (maps to server-side session data)

String Tokenisation — Concepts

In computing, tokenisation splits a string into meaningful units (tokens) for parsing, processing, or model input.

- Lexer tokenisation: break source code into language tokens (keywords, operators, literals).
- NLP tokenisation: split text into words, subwords, or characters for ML models.
- Delimiter-based: split(',') — simplest form.
- Byte Pair Encoding (BPE): merge frequent character pairs into subword units (GPT, LLaMA).

Text: "unhappiness" Character tokens: u n h a p p i n e s s (11 tokens) Word token: unhappiness (1 token) BPE subword: un + happi + ness (3 tokens) ← typical LLM output

UUIDs, ULIDs & NanoIDs — Identifier Standards

Unique identifiers are used as primary keys, token IDs, and correlation IDs across distributed systems.

- UUID v4: 128-bit random — universal, but no natural sort order.
- UUID v7: UUID with timestamp prefix — sortable, great for DB primary keys.
- ULID: 26-char base32, timestamp-prefixed, lexicographically sortable.
- NanoID: URL-safe, compact (21 chars default), cryptographically random.
- Use UUIDs/ULIDs for public-facing IDs; never expose sequential integer IDs.

UUID v4: 550e8400-e29b-41d4-a716-446655440000 (random) UUID v7: 018fce17-b8a1-7f5e-9c2d-3a4b5c6d7e8f (time-sortable) ULID: 01ARZ3NDEKTSV4RRFFQ69G5FAV (sortable) NanoID: V1StGXR8_Z5jdHi6B-myT (compact)

PART 6 — LLM Tokenisation & Model Comparison

How LLMs Tokenise Text

Large Language Models don't process raw characters or words — they operate on tokens, which are subword units produced by a trained tokeniser.

- Tokenisers compress vocabulary: ~100k chars → ~30–200k token vocabulary.
- Same word can produce different token counts across models.
- Token count affects: cost (billed per token), context window usage, speed.
- Punctuation, spaces, and case all affect tokenisation.
- Rule of thumb: 1 token ≈ 0.75 English words, or ~4 characters.

"ChatGPT is amazing!" tokenised (GPT-4o): ["Chat", "G", "PT", " is", " amazing", "!"] → 6 tokens "LLM" → 1 token | "tokenization" → 3 tokens "■" → 3 tokens | Chinese chars → 1-2 tokens each

■ Fewer tokens = lower API cost + more context available for reasoning.

Major LLMs — Tokenisation & Architecture Comparison

Model	Tokeniser	Vocab Size	Context Window	Strength
GPT-4o (OpenAI)	cl100k_base (tiktoken BPE)	~100,277	128K tokens	Multimodal, coding, reasoning, vision
GPT-4.1 (OpenAI)	o200k_base (tiktoken BPE)	~200,019	1M tokens	Long-context tasks, instruction follow
Claude 3.5/4 (Anthropic)	Custom BPE (sentencepiece)	~100K+	200K tokens	Long docs, analysis, coding, safety
Gemini 1.5/2 (Google)	SentencePiece Unicode BPE	~256K	1M–2M tokens	Ultra-long context, multimodal, search
LLaMA 3.x (Meta)	tiktoken cl100k_base	~128,256	128K tokens	Open-source, fine-tunable, on-device
Mistral/Mixtral (Mistral AI)	SentencePiece BPE	~32,000	32K–128K	Efficient, fast, MoE architecture
Grok 3 (xAI)	Custom BPE	~100K+	128K tokens	Real-time X data, coding, reasoning

Model	Tokeniser	Vocab Size	Context Window	Strength
Command R+ (Cohere)	Custom BPE (sentencepiece)	~256K	128K tokens	RAG, enterprise search, multilingual

Byte Pair Encoding (BPE) — How It Works

BPE is the dominant tokenisation algorithm used by GPT, LLaMA, and most modern LLMs. It learns a vocabulary by iteratively merging frequent character pairs.

- Step 1: Start with individual characters as tokens.
- Step 2: Count all adjacent pairs and merge the most frequent.
- Step 3: Repeat until vocabulary size is reached.
- Result: common words become single tokens; rare words split into subwords.
- Advantage: handles any language and unknown words gracefully.

Training BPE (simplified): Corpus: "low lower lowest" Start: l o w | l o w e r | l o w e s t
Merge "l"+"o" → "lo": lo w | lo w e r | lo w e s t Merge "lo"+"w" → "low": low | low e r | low e s t
Final vocab includes: low, lower, est, ##er, ##est

Context Window & Token Limits

The context window is the maximum number of tokens an LLM can process in one call — input + output combined.

- Larger context = more document, history, or code the model can 'see'.
- Exceeding the limit truncates early content (model forgets the start).
- Cost is proportional to tokens processed; use chunking for large docs.
- Prompt + completion must fit within the window.

Context window examples (approximate): GPT-3.5 → 16K tokens (~12K words) GPT-4o → 128K tokens (~96K words) Claude 3.5 → 200K tokens (~150K words) Gemini 1.5 Pro → 1M tokens (~750K words) GPT-4.1 → 1M tokens (~750K words)

■ 1M token context can fit ~800,000 words — roughly 10 full novels.

Tokenisation Differences Across LLMs

The same text produces different token counts across models due to different vocabularies and training corpora — this matters for cost and prompt design.

- GPT-4o uses cl100k_base; GPT-4.1 uses o200k_base (larger vocab = fewer tokens for code/multilingual).
- Claude's tokeniser is optimised for longer English documents.
- Gemini uses a Unicode-aware SentencePiece model — efficient for non-Latin scripts.
- Code tokens: Python keywords often single tokens; whitespace-sensitive languages cost more.
- Emojis and special chars are expensive — 2–6 tokens each in most tokenisers.

Text: "def calculate_total(items):" GPT-4o (cl100k): ["def", " calculate", "_total", "(", "items", "):"] = 6 LLaMA 3 (same tokeniser) = 6 Mistral (32k vocab): = 8 (splits differently)
"Bonjour" → GPT-4o: 1 token | older GPT-3: 3 tokens